

CodeAtlas: A Graph-Based Repository Understanding System for Software Comprehension and Graph-Grounded AI Assistance

Fatma Nageh, Mariam Ashraf, Tarek Mostafa, Youssef Hussein

*Faculty of Computer Science
Misr International University, Cairo, Egypt*

Dr. Fatma Helmy, Eng. Asmaa Maher

*Faculty of Computer Science
Misr International University, Cairo, Egypt
{fatma.helmy, asmaa.maher}@miuegypt.edu.eg*

Abstract—Understanding large software repositories becomes more challenging because important relationships among files, functions, classes, configuration files, documentation, and dependencies are often hidden across many disconnected artifacts. Developers usually depend on file explorers, keyword search, symbol navigation, static call views, and fragmented documentation to answer repository-level questions such as what depends on a component, where a feature is implemented, or what may be affected by a change. These tools are useful for local navigation, but they do not always provide a persistent and clear picture of the repository as a connected system.

This paper presents CodeAtlas, a repository understanding system that transforms a software codebase into an interactive knowledge graph. CodeAtlas scans a selected repository, classifies files, parses supported source-code files using Tree-sitter, normalizes extracted facts into an intermediate representation, stores repository entities and relationships in Neo4j, and exposes the graph through a web interface and a Visual Studio Code extension. The system also supports graph-grounded question answering by retrieving relevant graph nodes, relationships, paths, and text chunks using GraphRAG before generating AI-assisted explanations. This design aims to improve software comprehension, technical onboarding, debugging, refactoring preparation, and impact analysis by making repository structure explicit and explainable.

I. INTRODUCTION

Modern software repositories contain hundreds of interconnected source files, source code, tests, documentation, configuration, and scripts. Understanding how these pieces relate to each other is essential as a matter of fact its crucial for technical onboarding, debugging, and impact analysis, yet existing tools present them in isolation. Regardless of the existing traditional tools used, Developers are going to struggle to gather the full picture of how a complete software system is structured.

CodeAtlas addresses this gap by transforming a codebase into an interactive knowledge graph where every entity and their relationships are explicit, explorable, and queryable. The system provides a unified interface that serves both as a navigation layer for developers and as a grounding layer for

AI-assisted explanations, reducing the effort needed to form an accurate mental model of the repository.

We present the design of CodeAtlas, its graph schema, and its indexing methodology. An evaluation across repositories of varying scale examines graph construction quality, extraction accuracy, performance, and the effectiveness of graph-grounded AI assistance. With CodeAtlas there would be improvements in comprehension task efficiency compared to traditional file-level navigation.

The main contribution of CodeAtlas is not only visualization. It combines repository scanning, parser-based extraction, graph persistence, interactive exploration, and graph-grounded AI assistance into one workflow. The graph becomes both a navigation layer for humans and a grounding layer for AI responses. When a user asks a question, CodeAtlas retrieves relevant graph context before generating an answer. This reduces the risk of unsupported explanations compared with ungrounded prompting.

This paper makes five main contributions:

- A compact repository knowledge graph schema for representing source-code, documentation, and structural relationships.
- A repository indexing pipeline based on file classification, Tree-sitter parsing, intermediate representation, and Neo4j ingestion.
- A developer-facing exploration workflow through a web interface and a VS Code extension.
- A GraphRAG-style assistance approach that retrieves graph and text evidence before generating explanations.
- An evaluation framework covering graph validity, extraction accuracy, performance, AI groundedness, and usability.

II. RELATED WORK

A. Software Comprehension and Developer Technical Onboarding

Software comprehension research shows that developers understand systems by forming mental models of how artifacts

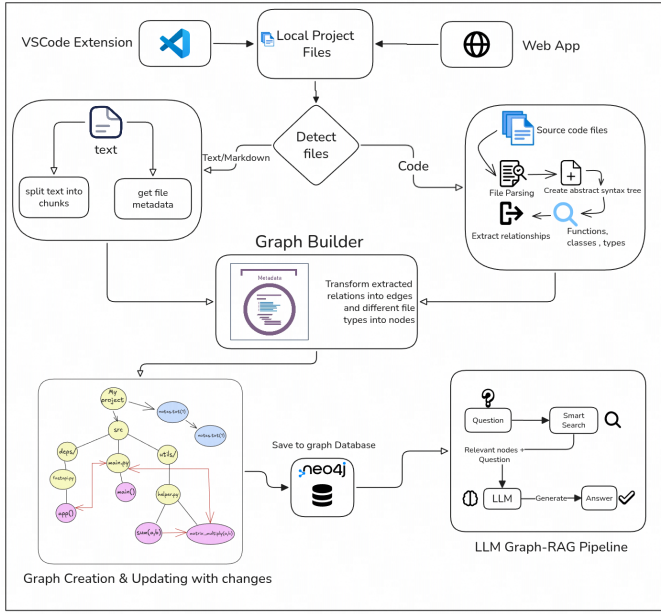


Fig. 1. CodeAtlas system overview diagram.

relate to each other, not by reading isolated files only. This is especially important during technical onboarding, where newcomers need to understand architecture, naming conventions, module boundaries, and feature locations before making safe changes. Existing technical onboarding tools support specific activities, but many still require users to manually connect information from code, documentation, and development history [1].

CodeAtlas builds on this need by making repository relationships explicit. Instead of only showing individual files or search results, it offers a connected view of repository artifacts. This is useful for beginners who need a high-level map and experienced developers who need to inspect dependency paths or impact areas.

B. Software Visualization and Architecture Recovery

Software visualization and architecture recovery tools aim to expose system structure through diagrams, dependency graphs, call graphs, or module views. These tools help developers move from local code reading to system-level understanding. However, visualizations can become outdated, visually dense, or disconnected from the developer’s daily workflow. A graph with too many nodes can also become difficult to interpret unless it supports search, filtering, progressive disclosure, and node-level inspection.

CodeAtlas follows the graph visualization direction but adds persistent storage and AI-assisted explanation. The goal is not to render every possible relationship at once. Instead, the system supports bounded neighborhoods, search, filtering, and evidence-based summaries so users can move from overview to detail gradually.

C. Code Knowledge Graphs and Static Analysis

Code knowledge graphs model software entities and their relationships as structured graph data. Prior work on semantic code graphs and repository-level graph navigation shows that graph representations can support comprehension, dependency analysis, and AI-assisted software engineering [2]–[4]. Static analysis is commonly used in these systems because it can extract useful structural information without executing the project.

CodeAtlas adopts this direction through Tree-sitter-based parsing and a normalized intermediate representation. Tree-sitter is suitable because it provides incremental parsing and concrete syntax trees supporting many programming languages [5]. However, CodeAtlas also recognizes the limits of static analysis. Dynamic imports, reflection, dependency injection, generated code, and framework-managed behavior may not be fully captured without runtime analysis or language-specific semantic tooling.

D. RAG and GraphRAG for Software Engineering

Retrieval-Augmented Generation (RAG) improves language-model answers by retrieving relevant external context before generation. For software repositories, retrieved context may include source files, documentation chunks, dependency information, or issue descriptions. Text-only retrieval is useful, but it may miss structural connections when related files do not share similar wording.

GraphRAG extends retrieval by using graph structure as part of the grounding process [7]. This is important for software because codebases are naturally relational. Files import other files, functions call functions, classes extend classes, and documentation describes code elements. CodeAtlas applies this idea by retrieving nodes, relationships, paths, and text chunks from the repository graph before generating an answer. By gathering relevant context, the system reduces hallucinations and keeps AI responses more related to the developer’s prompt. The system treats AI output as assistance, not proof; therefore, retrieved evidence should remain inspectable by the user.

III. METHODOLOGY

A. Dataset

CodeAtlas is evaluated using real-world software repositories rather than a traditional fixed machine-learning dataset. In this research, each repository acts as an input dataset. The dataset contains source files, documentation files, configuration files, directories, parsed code entities, text chunks, and relationships extracted from the repository structure.

The repository dataset is important because CodeAtlas does not work on isolated text samples. It works on connected software artifacts. Each selected repository is scanned and transformed into a graph representation. The graph then becomes the base for visualization, analysis, querying, and graph-grounded AI assistance.

For a fair evaluation, the dataset should include repositories of different sizes and structures. A small repository can be used for manual validation because its files and relationships

TABLE I
SUBJECT REPOSITORY DATASET PROFILE

Repository Type	Purpose
Small repository	Controlled validation of extracted nodes and relationships.
Medium repository	Realistic graph construction and user task testing.
Large/polyglot repository	Scalability, parsing coverage, and graph density testing.
AI-oriented repository	Testing documentation-code links and graph-grounded questions.

are easier to inspect. A medium repository can be used to test realistic development scenarios. A large or multi-language repository can be used to test scalability, parsing coverage, and graph density. An AI-oriented repository can also be used to test documentation-code links and graph-grounded question answering.

For reproducibility, each repository should be reported with its exact commit hash, main languages, file count, supported code-file count, supported text-file count, ignored-file count, generated node count, generated relationship count, and indexing time. This makes the experiment easier to repeat and compare.

B. Used Algorithms

The CodeAtlas algorithm is based on a multi-stage repository indexing and graph construction pipeline. The system starts by scanning the repository, then classifies files, parses supported source-code files, converts extracted facts into an intermediate representation, stores the final graph in Neo4j, and finally retrieves graph evidence for AI-assisted answers.

1) Abstract Syntax Tree (AST) Parsing and Intermediate Representation:

- First, the scanner recursively traverses the repository and records metadata such as relative path, extension, size, modification time, and detected language. High-noise directories such as `.git`, dependency folders, build outputs, caches, binaries, and generated artifacts are ignored to reduce graph noise and indexing cost.
- Second, supported source-code files are parsed using Tree-sitter. The extraction layer traverses syntax trees and extracts facts such as declarations, imports, AST parent-child relationships, source locations, and parse diagnostics. These facts are not inserted directly into the database. Instead, they are converted into a language-neutral intermediate representation.
- Third, the intermediate representation works as the normalization boundary between parsing and graph storage. It assigns stable identifiers, normalized labels, relationship types, source locations, and properties. Deterministic identifiers are important because they support repeated indexing and incremental updates.

For example, a function node can be identified using repository ID, file path, node type, qualified name, and source location.

2) Knowledge Graph Construction and Graph Modeling:

- Fourth, the system stores repository entities and relationships in Neo4j. Neo4j is suitable because repository comprehension is relationship-centered. CodeAtlas uses uniqueness constraints and indexes on stable identifiers to reduce duplicate nodes and improve query performance. Typical queries include repository overview, node details, bounded neighborhoods, path discovery, graph search, and GraphRAG context retrieval. The graph schema is designed to preserve both repository hierarchy and semantic relationships.

• Graph Relationship such as:

- CONTAINS
- IMPORTS
- REFERENCES
- EXTENDS
- DECLARES
- HAS_AST_ROOT
- HAS_CHUNK
- DESCRIBES

3) Embedding Generation and Semantic Retrieval:

- Fifth, CodeAtlas uses vector embeddings to support semantic search and GraphRAG-based retrieval. Source files, AST nodes, documentation chunks, and repository summaries are transformed into dense vector representations using embedding models. The embedding pipeline enables semantic similarity retrieval beyond traditional keyword matching.
- CodeAtlas uses an Embedding Model (text-embedding-3-small) for embedding generation. The generated embeddings are stored alongside graph entities and used during retrieval operations.
- Semantic retrieval enables CodeAtlas to perform context-aware repository exploration by retrieving entities that are both semantically and structurally relevant to the developer's query. Instead of relying solely on traditional keyword-based search, the system uses embedding similarity to identify related files, functions, and documentation fragments, while graph traversal is used to expand the retrieved context through dependency relationships and architectural connections.

4) GraphRAG Pipeline:

Finally, the GraphRAG pipeline grounds AI responses in repository evidence. It retrieves relevant graph nodes,

Algorithm 1 Graph-Grounded Repository Question Answering

Require: User question q , repository graph G , text chunks T
Ensure: Grounded answer with supporting evidence

- 1: Normalize q and identify likely intent
- 2: Retrieve candidate graph nodes from G
- 3: Expand bounded neighborhoods around candidates
- 4: Retrieve relevant text chunks from T
- 5: Rank graph and text evidence by relevance and graph distance
- 6: Build a prompt using only selected evidence
- 7: Generate an answer using the configured language model
- 8: Return answer with links to files, nodes, paths, or chunks

expands bounded neighborhoods, retrieves related text chunks, ranks the evidence, and uses the selected context to generate a grounded answer.

The pipeline is useful because repository questions often require both semantic and structural context. For example, an impact-analysis question may need imported files, declared entities, dependent files, and related documentation. GraphRAG can retrieve this connected context more directly than text-only retrieval. However, the system does not claim that GraphRAG eliminates hallucination. It only improves grounding when the retrieved evidence is relevant and complete.

TABLE II
REPOSITORY FILE CATEGORIES

Category	Meaning
CodeFile	Supported source-code file parsed structurally.
TextFile	Documentation or text file chunked for retrieval.
IgnoredFile	File excluded by ignore rules, such as build output or dependencies.
UnsupportedFile	Discovered file that is not processed structurally.

C. Performance Metrics

The evaluation of CodeAtlas is based on metrics that measure graph construction quality, extraction accuracy, system performance, GraphRAG quality, and usability. These metrics are selected because CodeAtlas is not only a visualization tool. It is also an indexing, analysis, retrieval, and AI-grounding system.

Graph construction metrics show whether CodeAtlas can build a valid and clean repository graph. Extraction accuracy metrics show whether the parsed entities and relationships match the real codebase. Performance metrics show whether the system can process repositories in a practical time. GraphRAG quality metrics show whether the generated answers are supported by retrieved evidence. Usability metrics show whether developers can use the system effectively for repository comprehension tasks.

The evaluation is organized around five research questions:

TABLE III
CORE KNOWLEDGE GRAPH SCHEMA

Node/Edge	Role
Repo	Root node for the indexed repository.
Directory	Folder in the repository hierarchy.
CodeFile	Parsed source-code file.
TextFile	Documentation or supported text artifact.
TextChunk	Segment extracted from a text file.
AstNode	Parsed code entity or syntax element.
CONTAINS	Repository or directory containment relation.
DECLARES	Connects a code file to declared code entities.
IMPORTS	Represents import/include relationships where resolvable.
AST_CHILD	Represents syntax-tree hierarchy.
DESCRIBES/MENTIONS	Connects documentation chunks to related code artifacts.

TABLE IV
PERFORMANCE METRICS

Metric Group	Examples
Graph construction	Node count, relationship count, schema validity, duplicate nodes, orphan nodes.
Extraction accuracy	Precision, recall, missing entities, incorrect relationships, error categories.
Performance	Scanning time, parsing time, ingestion time, full indexing time, query latency.
GraphRAG quality	Retrieval relevance, answer correctness, groundedness, hallucination rate, evidence coverage.
Usability	Task completion, completion time, confidence, perceived usefulness, qualitative feedback.

- **RQ1:** Can CodeAtlas construct a valid repository knowledge graph?
- **RQ2:** How accurately does CodeAtlas extract entities and relationships?
- **RQ3:** How efficient is indexing, updating, searching, and traversal?
- **RQ4:** Does graph-grounded retrieval improve answer relevance and groundedness compared with text-only retrieval?
- **RQ5:** How useful and usable is CodeAtlas for repository comprehension tasks?

IV. EXPERIMENTAL RESULTS

The experimental results are designed to compare CodeAtlas with realistic developer workflows. The first baseline is manual exploration using the VS Code file explorer and keyword search. The second baseline is traditional IDE navigation such as symbol lookup and go-to-definition. The third baseline is text-only RAG, where answers are generated from retrieved text chunks without graph traversal. The fourth baseline is an LLM-only answer without repository-specific graph grounding. These baselines help show whether the graph adds value beyond normal search and unstructured AI assistance.

Based on the system design, CodeAtlas is expected to perform best in tasks that depend on relationships. Traditional keyword search is strong when the user already knows the

term to search for. However, graph exploration is stronger when the user needs to understand connections between files, modules, functions, documentation, and dependencies. This makes CodeAtlas suitable for onboarding, impact analysis, architecture inspection, and debugging preparation.

The graph representation is expected to improve repository understanding by showing direct relationships between artifacts. For example, a developer can start from a file, inspect its declared functions, view its imports, follow related documentation, and explore connected files. This is more useful than reading isolated files because it shows how the system parts are connected.

The GraphRAG component is expected to improve AI answer groundedness compared with LLM-only answers. Instead of generating explanations from model knowledge alone, CodeAtlas retrieves graph and text evidence first. Users can inspect the retrieved nodes, relationships, and files to judge whether the answer is supported. This makes the AI assistance more transparent and more reliable.

The expected practical impact of CodeAtlas lies in reducing the effort required to build an initial mental model of a repository. A developer can start from a high-level graph, search for a feature, inspect related files, view connected documentation, and ask targeted questions. This workflow can support technical onboarding, maintenance planning, and safer refactoring decisions.

If the experimental sample is small, the results should be reported as preliminary feedback rather than statistically generalizable evidence. This avoids overclaiming and keeps the research academically defensible.

V. CHALLENGES AND LIMITATIONS

A. Scalability

Large repositories can produce thousands of files, many parsed entities, dense relationships, and large text-chunk sets. This increases scanning time, parsing cost, Neo4j ingestion time, query latency, and graph rendering complexity. CodeAtlas should therefore use incremental updates, caching, parallel parsing, subgraph partitioning, bounded traversal, and lazy graph expansion.

B. Static Analysis Limits

CodeAtlas relies mainly on static parsing. This is effective for explicit declarations, imports, and syntax structure, but weaker for runtime-only behavior. Dynamic imports, reflection, dependency injection, generated code, and framework conventions may introduce relationships that are missing from the static graph. Future work should integrate execution traces, logs, framework-specific extractors, or build-system analysis.

C. GraphRAG and LLM Limits

GraphRAG improves grounding only when retrieval succeeds. If the retrieval layer misses relevant nodes, retrieves noisy context, or uses traversal depths that are too shallow, the generated answer may still be incomplete. Large repositories may also exceed model context limits. To reduce these risks,

CodeAtlas should use context pruning, confidence scoring, evidence links, answer verification, and local-model support for privacy-sensitive repositories.

D. Threats to Validity

Internal validity may be affected by extraction bugs, incomplete parser support, and manual labeling errors during evaluation. Construct validity may be affected if the selected metrics do not fully capture repository comprehension. External validity may be limited if the subject repositories do not represent all repository types, languages, or development practices. Conclusion validity may be limited by small user-study samples. These threats can be reduced by reporting repository versions, using multiple repositories, defining clear scoring rubrics, keeping evaluation scripts available, and avoiding unsupported generalizations.

VI. CONCLUSION AND FUTURE WORK

This paper presented CodeAtlas, a graph-based repository understanding system for software comprehension and graph-grounded AI assistance. CodeAtlas transforms repository artifacts into a knowledge graph using scanning, classification, Tree-sitter parsing, intermediate representation, and Neo4j storage. It then exposes the graph through developer-facing interfaces and uses retrieved graph evidence to support AI-assisted explanations.

The main value of CodeAtlas is that it makes repository relationships explicit. Instead of relying only on isolated file views or ungrounded AI summaries, developers can explore connected repository structure, inspect nodes and relationships, trace paths, and ask questions grounded in graph evidence. This supports onboarding, debugging, refactoring preparation, and impact analysis.

Future work should expand language and framework support, improve incremental updates, add runtime evidence, strengthen graph visualization for large repositories, and evaluate CodeAtlas across more repositories and users. The system can also be extended with architectural smell detection, duplicate logic discovery, refactoring recommendations, collaborative annotations, and stronger graph-native retrieval methods.

REFERENCES

- [1] J. Santos, I. Steinmacher, and M. A. Gerosa, "Software solutions for newcomers onboarding in software projects: A systematic literature review," *arXiv preprint*, 2024.
- [2] A. E. Hassan and R. C. Holt, "A semantic code graph model for software comprehension and analysis," in *Proceedings of Software Maintenance and Evolution Research*, 2023.
- [3] Z. Liu, Y. Wang, and H. Zhang, "CodexGraph: Bridging large language models and code repositories via graph database interfaces," in *Proceedings of the Association for Computational Linguistics*, 2024.
- [4] Y. Ouyang, J. Li, and X. Chen, "RepoGraph: Enhancing AI software engineering with repository-level code graphs," *arXiv preprint*, 2025.
- [5] Tree-sitter, "Tree-sitter: An incremental parsing system for programming tools," 2025. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [6] Neo4j, "Neo4j graph database documentation," 2025. [Online]. Available: <https://neo4j.com/docs/>
- [7] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

- [8] X. Chen, Y. Zhou, and L. Wang, "LocAgent: Graph-guided LLM agents for code localization," *arXiv preprint*, 2025.
- [9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [10] ISO/IEC, "ISO/IEC 25010: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models," International Organization for Standardization, 2011.
- [11] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, Taylor and Francis, 1996.
- [12] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," EBSE Technical Report, 2007.